LAPORAN PRATIKUM PEMROGRAMAN

BERORIENTASI OBJEK MINGGU KE-4



Disusun oleh:

Fauzan Fadillah (2515511038)

2B/D3

Repository:

https://github.com/MiawGit/PBO_W4_038.git

1. Bagian A

1) Code Inti

```
package Task1.source;

public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height
             + 2 * Math.PI * getRadius() * getRadius();
    }

    public double getVolume() {
        return super.getArea() * height;
    }

    @Override
    public String toString() {
        return "Cylinder: subclass of " + super.toString()
             + ", height=" + height;
    }
}
```

Penjelasan:

Kode ini menerapkan konsep pewarisan (inheritance) melalui syntax extends circle, yang memungkinkannya mewarisi sifat kelas induk sekaligus menambahkan atribut baru berupa tinggi (height). Pada bagian konstruktor, program ini memanggil super() atau super(radius) untuk memastikan bahwa objek superclass dibangun dan diinisialisasi terlebih dahulu sebelum memproses variabel milik subclass.

2) Output TestCylinder

```
PS C:\Users\fauza\OneDrive\Documents\KULIAH\SEMESTER 3\PBO P\W4\251511038_Fauzan_Fadillah_PBO4> & 'c:\Users\fauza\.vscode\extens
ions\redhat.java-1.57.2026093008-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp
' 'C:\Users\fauza\AppData\Roaming\Code\User\workspaceStorage\344f1a64484c32f29c395de7fb7beb77\redhat.java\jdt_ws\251511038_Fauzan
_Fadillah_PBO4_15fd79e6\bin' 'Task1.source.TestCylinder'
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172

Object c3:
Cylinder: subclass of Circle[radius=2.0, color=red], height=10.0
PS C:\Users\fauza\OneDrive\Documents\KULIAH\SEMESTER 3\PBO P\W4\251511038_Fauzan_Fadillah_PBO4>
```

2. Bagian B

1. Code Inti

    1. Shape.java

```java
public class Shape {
    private String color;
    private boolean filled;

    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }
```

```
    public boolean isFilled() {
        return filled;
    }

    public void setFilled(boolean filled) {
        this.filled = filled;
    }

    @Override
    public String toString() {
        return "Shape   [color = " + color + ", filled = " + filled
   + "]";
    }
}
```

2. Square.java

```
public class Square extends Rectangle {

    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setWidth(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setLength(double side) {
```

```
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square  [side=" + getSide()
             + ", " + super.toString() + "]";
    }
}
```

2. Output

```
Shape   [color = black, filled = false]
Circle      [radius=2.0, Shape   [color = blue, filled = true]] area = 12.566370614359172
Rectangle   [width = 3.0, length = 4.0, Shape   [color = yellow, filled = true]] area = 12.0
Square  [side=5.0, Rectangle   [width = 5.0, length = 5.0, Shape   [color = green, filled = true]]] area = 25.0

=== Test Square Setter ===
Awal         : Square  [side=5.0, Rectangle   [width = 5.0, length = 5.0, Shape   [color = green, filled = true]]]
setWidth(8)  : Square  [side=8.0, Rectangle   [width = 8.0, length = 8.0, Shape   [color = green, filled = true]]]
setLength(3) : Square  [side=3.0, Rectangle   [width = 3.0, length = 3.0, Shape   [color = green, filled = true]]]
Area         : 9.0
Perimeter    : 12.0
PS C:\Users\fauza\OneDrive\Documents\KULIAH\SEMESTER 3\PBO P\W4\251511038_Fauzan_Fadillah_PBO4>
```

3. Bagian C

1. Alasan Memilih Inhertance

   Inheritance dipilih karena terdapat relasi 'Is-A' yang jelas. Manager dan Staff adalah seorang Employee. Oleh karena itu, atribut dasar seperti nama, gaji pokok, dan tahun masuk ditarik ke dalam superclass Employee. Sementara itu, Manager dan Staff dibuat sebagai subclass karena memiliki spesifikasi (behavior) yang berbeda, misalnya Manager memiliki perhitungan bonus gaji, sedangkan Staff memiliki jabatan spesifik.

2. Code Inti

   1. Employee.java

```
package KasusSebelumnya;

public class Employee {
    private String name;
    private double baseSalary;
    private int hireYear;

    public Employee(String name, double baseSalary, int hireYear) {
        this.name = name;
        this.baseSalary = baseSalary;
```

```
        this.hireYear = hireYear;
    }

    public String getName() {
        return name;
    }

    public double getSalary() {
        return baseSalary;
    }

    public int getHireYear() {
        return hireYear;
    }

    public String getDetails() {
        return "Nama: " + name + ", Gaji Pokok: " + baseSalary + ",
Tahun Masuk: " + hireYear;
    }
}
```

2. Manager.java

```
package KasusSebelumnya;

public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double baseSalary, int hireYear,
double bonus) {
        super(name, baseSalary, hireYear);
        this.bonus = bonus;
    }

    public void setBonus(double bonus) {
        this.bonus = bonus;
    }

    @Override
    public double getSalary() {
        return super.getSalary() + bonus;
    }

    @Override
    public String getDetails() {
        return "Manager [" + super.getDetails() + ", Bonus: " +
bonus + ", Total Gaji: " + getSalary() + "]";
    }
}
```

3. Output Pengujian

```
--- Data Karyawan ---
Manager [Nama: Antonio Rossi, Gaji Pokok: 2.9E7, Tahun Masuk: 1989, Bonus: 5000000.0, Total Gaji: 3.4E7]
Staff [Nama: Maria Bianchi, Gaji Pokok: 2500000.0, Tahun Masuk: 1991, Posisi: IT Support]

--- Pengecekan Method Warisan ---
Tahun masuk Manager : 1989
Tahun masuk Staff   : 1991
PS C:\Users\fauza\OneDrive\Documents\KULIAH\SEMESTER 3\PBO P\W4\251511038_Fauzan_Fadillah_PBO4>
```

4. Refleksi

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?

Karena Circle mewakili bangun datar dua dimensi dasar yang hanya bergantung pada satu state (jari-jari). Sedangkan Cylinder adalah bangun ruang tiga dimensi yang lebih spesifik karena menambahkan state tinggi (height). Setiap silinder pasti memiliki alas berbentuk lingkaran, sehingga Circle sangat logis dijadikan kelas induk yang lebih umum.

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?

Arti praktisnya adalah code reusability. Objek Cylinder secara otomatis mewarisi method seperti getRadius() dan getArea() dari Circle tanpa kita harus repot-repot mengetik ulang kodenya dari awal di dalam file Cylinder.java.

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?

Karena di dalam konsep pewarisan, objek superclass (Circle) yang menjadi fondasi dasar harus dibangun dan diinisialisasi terlebih dahulu di memori sebelum subclass spesifiknya (Cylinder) bisa menambahkan atribut-atribut barunya (seperti height).

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?

Fungsi getVolume() aslinya dihitung dari getArea() * height. Setelah getArea() di-override untuk menghitung "luas permukaan tabung", maka perkalian tersebut bukan lagi mengalikan luas alas dengan tinggi, melainkan mengalikan luas

permukaan dengan tinggi, yang menyebabkan perhitungan matematika volumenya jadi salah (nge-bug).

5. **Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?**

   Saya secara eksplisit meminta program (compiler) untuk mengabaikan fungsi override yang ada di dalam subclass saat ini, dan mengeksekusi metode aslinya yang terdapat di dalam superclass.

6. **Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?**

   Karena bujur sangkar (Square) memiliki aturan mutlak di mana panjang dan lebarnya harus selalu sama ukurannya. Jika *setter* dari Rectangle tidak diubah, maka panjang dan lebarnya bisa diubah dengan angka yang berbeda secara terpisah, yang akan merusak keabsahan objek bujur sangkar tersebut.

7. **Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.**

   Tidak harus, karena inheritance hanya boleh dipakai jika kedua class tersebut memiliki relasi logis "Is-A" (Adalah Sebuah). Contohnya Manager adalah sebuah Employee, sehingga sangat cocok memakai inheritance. Jika dua class hanya mirip nama variabelnya tapi tidak ada hubungan induk-anak, lebih baik menggunakan metode Composition.

8. **Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."**

   Inheritance bukan sekadar mengurangi duplikasi, tetapi tentang membangun hierarki objek yang logis secara konseptual, di mana *subclass* mewarisi sifat dasar *superclass*-nya, sambil tetap memiliki kendali untuk memodifikasi perlakuannya melalui fungsi polimorfisme (*overriding*).